Fosol-Cycle
Decision Algorithm & Coding Implementation Guide

For the Decision Logic / Algorithm Lead

Using the project-ready NASA climate JSON + project-ready Rajshahi crop JSON

Core idea: First eliminate choices that violate hard constraints. Then classify the proposed rotation. Only after that should the system score and rank the remaining feasible alternatives.

Farmer / field input

↓

Crop JSON + NASA climate JSON

↓

Window Guard: SAFE / COMPRESSED / OUTSIDE SUPPORTED WINDOW

↓

Biophysical checks: temperature / soil / water

↓

Rotation GPS: test all candidate crops

↓

Rank feasible candidates using farmer priorities

↓

Return an explainable recommendation

Important: this guide intentionally avoids inventing exact weather-to-delay rules that your current evidence does not validate.

1. What the Decision Engine Receives

Input	Examples	What it is used for
Farmer / field input	Current crop, planting date, next crop, irrigation, soil pH, priorities	Defines the actual farm scenario
Crop JSON	Planting windows, variety duration, pH range, family, water context, source confidence	Defines crop requirements and constraints; generated from the editable research CSV
NASA climate JSON	Daily temperature, rainfall, surface wetness, humidity, historical context	Defines the environmental scenario

The decision engine should never treat these three inputs as interchangeable. The crop JSON tells you what the crop needs; NASA tells you what the environment did; the farmer input tells you the local plan and priorities. Keep the researched CSV as the editable master, but convert it to JSON before the application uses it.

2. Status Vocabulary

Internal status	Meaning	Recommended UI
SAFE	Field can be ready inside the preferred planting window and hard constraints pass.	Green - Safe
COMPRESSED	Preferred window is partly or fully lost, but a documented supported-late window still remains.	Yellow - Compressed
OUTSIDE_SUPPORTED_WINDOW	Field-ready date is later than the latest documented window in the dataset.	Red - Outside supported window
UNKNOWN	A required rule/value is missing or confidence is too low.	Gray - Unknown

Why not call everything MISSED?: Your current crop dataset deliberately has hard_deadline_validated = FALSE. A recommended planting-window end is not automatically a biological hard cutoff. Internally keep OUTSIDE_SUPPORTED_WINDOW; the UI may explain that the supported window has been missed.

3. Full Decision Algorithm

1. Validate farmer input and crop IDs

↓

2. Load the current crop profile

↓

3. Calculate current-crop harvest range

↓

4. Add turnaround / preparation time -> field-ready range

↓

5. Build the next crop's planting window for the correct year

↓

6. Evaluate the planting window -> SAFE / COMPRESSED / OUTSIDE

↓

7. Evaluate temperature / soil / water / location constraints

↓

8. Attach NASA historical risk flags

↓

9. If plan is compressed/outside, evaluate every alternative crop

↓

10. Hard-filter infeasible candidates

↓

11. Score the survivors by soil / water / risk / income

↓

12. Apply farmer-priority weights

↓

13. Return ranked alternatives + reasons + confidence

3.1 Step 1 - Validate Inputs

- Current crop ID must exist in the crop JSON.
- Proposed next crop ID must exist.
- Planting/transplanting date must be a valid ISO date.

- Turnaround days must be supplied until local expert rules are validated.
- Soil pH and irrigation can be optional; missing values should produce UNKNOWN/PARTIAL checks, not fabricated values.
- Farmer priority weights should be bounded, e.g. 1 to 5.

```
const validateInput = (input, cropsById) => {
  if (!cropsById[input.currentCrop]) throw new Error("Unknown current crop");
  if (!cropsById[input.proposedNextCrop]) throw new Error("Unknown next crop");
  if (Number.isNaN(Date.parse(input.plantingDate))) throw new Error("Invalid planting date");
  if (input.turnaroundDays == null) throw new Error("Turnaround days required for MVP");
  return true;
};
```

3.2 Step 2 - Calculate Harvest Range

Use the selected variety duration first. Do not use the generic EcoCrop duration when your custom dataset has a local/variety duration. If the dataset gives a range, preserve the range.

```
const addDays = (dateString, days) => {
  const d = new Date(dateString + "T00:00:00Z");
  d.setUTCDate(d.getUTCDate() + days);
  return d.toISOString().slice(0, 10);
};

const calculateHarvestRange = (plantingDate, crop) => ({
  earliest: addDays(plantingDate, crop.duration.variety.min_days),
  latest: addDays(plantingDate, crop.duration.variety.max_days)
});
```

Example: Aman BR11 transplanted on 2023-07-15 with a 145-day reference duration gives an estimated harvest around 2023-12-07. If a crop has 108-112 days, keep both earliest and latest dates.

3.3 Step 3 - Calculate Field-Ready Range

Field-ready date is not necessarily harvest date. The field may require harvesting, residue handling, drainage/drying and land preparation. Your dataset intentionally leaves exact turnaround rules blank because the evidence is not yet strong enough to automate them.

```
const calculateFieldReadyRange = (harvestRange, turnaroundDays) => ({
  earliest: addDays(harvestRange.earliest, turnaroundDays),
  latest: addDays(harvestRange.latest, turnaroundDays)
});
```

MVP rule: Ask the farmer/expert for turnaroundDays or use a clearly documented prototype assumption. Do NOT code WET = +5 days unless a local agronomic source validates that mapping.

3.4 Step 4 - Build the Next Crop Planting Window

Your crop JSON stores planting-window values as MM-DD strings. The algorithm must attach the correct calendar year. Some windows cross New Year, such as Boro.

```
const mmddToSeasonDate = (mmdd, seasonStartYear, crossesYear, isEnd = false) => {
  const [month, day] = mmdd.split("-").map(Number);
  let year = seasonStartYear;
  if (crossesYear && isEnd) year += 1;
  return `${year}-${String(month).padStart(2,"0")}-${String(day).padStart(2,"0")}`;
};

const buildPlantingWindow = (crop, seasonStartYear) => {
  const w = crop.planting_window;

  return {
```

```
    preferredStart: mmddToSeasonDate(
      w.preferred_start_mmdd, seasonStartYear, w.crosses_year, false
    ),
    preferredEnd: mmddToSeasonDate(
      w.preferred_end_mmdd, seasonStartYear, w.crosses_year, true
    ),
    lateEnd: w.supported_late_end_mmdd
      ? mmddToSeasonDate(
          w.supported_late_end_mmdd, seasonStartYear, w.crosses_year, true
        )
      : null
  };
};
```

3.5 Step 5 - Window Guard Classification

The window classifier is the heart of Fosol-Cycle. Use the latest plausible field-ready date for a conservative status. Preserve the earliest date as explanation/context.

```
const evaluateWindow = (fieldReadyRange, window) => {
  const ready = fieldReadyRange.latest;

  // If field is ready early, the farmer can wait until the preferred start.
  if (ready <= window.preferredEnd) {
    return {
      status: "SAFE",
      possiblePlantingDate: ready < window.preferredStart
        ? window.preferredStart
        : ready
    };
  }

  if (window.lateEnd && ready <= window.lateEnd) {
    return {
      status: "COMPRESSED",
      possiblePlantingDate: ready
    };
  }

  return {
    status: "OUTSIDE_SUPPORTED_WINDOW",
    possiblePlantingDate: ready
  };
};
```

Field-ready result	Preferred window	Late support	Status
Before preferred start	Still open	N/A	SAFE - wait until start
Inside preferred window	Open	N/A	SAFE
After preferred end	Closed	Still open	COMPRESSED
After supported late end	Closed	Closed	OUTSIDE_SUPPORTED_WINDOW
No late end in dataset and preferred missed	Closed	Unknown	OUTSIDE/UNKNOWN depending rule confidence

4. Climate and Biophysical Checks

Timeline fit is only Level 1. A crop may fit the calendar but still be unsuitable because of temperature, soil, water or location constraints. These checks should be separate so the result is explainable.

4.1 Historical Climate Context vs Crop Tolerance

Keep two ideas separate: (1) historically HOT/WET/DRY relative to Rajshahi history, and (2) biologically inside/outside the selected crop tolerance. A historically hot year can still be within the crop's temperature tolerance.

```
const getDailyRange = (climateJson, startDate, endDate) => {
  const out = [];
  const d = new Date(startDate + "T00:00:00Z");
  const end = new Date(endDate + "T00:00:00Z");

  while (d <= end) {
    const key = d.toISOString().slice(0, 10);
    if (climateJson.daily[key]) {
      out.push({ date: key, ...climateJson.daily[key].weather });
    }
    d.setUTCDate(d.getUTCDate() + 1);
  }
  return out;
};

const evaluateTemperature = (daily, crop) => {
  const tRule = crop.environment.temperature;
  const tolMin = tRule.tolerance_min_c;
  const tolMax = tRule.tolerance_max_c;
  const optMin = tRule.optimal_min_c;
  const optMax = tRule.optimal_max_c;

  let optimal = 0, tolerated = 0, outside = 0;

  for (const day of daily) {
    const t = day.temperature_c;
    if (t == null) continue;

    if (t >= optMin && t <= optMax) optimal++;
    else if (t >= tolMin && t <= tolMax) tolerated++;
    else outside++;
  }

  return {
    optimalDays: optimal,
    toleratedDays: tolerated,
    outsideToleranceDays: outside
  };
};
```

Interpretation: outsideToleranceDays > 0 is a strong warning, not automatically proof of crop failure. Your NASA value is gridded regional climate data and your crop tolerance values are generalized constraints.

4.2 Soil pH Check

```
const evaluateSoilPH = (soilPH, crop) => {
  if (soilPH == null) return { status: "UNKNOWN", score: null };
```

```
  const soil = crop.environment.soil;
  const optMin = soil.ph_optimal_min;
  const optMax = soil.ph_optimal_max;
  const tolMin = soil.ph_tolerance_min;
  const tolMax = soil.ph_tolerance_max;

  if (soilPH >= optMin && soilPH <= optMax) {
    return { status: "OPTIMAL", score: 100 };
  }

  if (soilPH >= tolMin && soilPH <= tolMax) {
    return { status: "TOLERATED", score: 60 };
  }

  return { status: "OUTSIDE_TOLERANCE", score: 0 };
};
```

The 100/60/0 values are a transparent project scoring convention, not a scientific measurement. Keep them documented and editable.

4.3 Water / Irrigation Check

Do not calculate irrigation demand as crop water requirement minus NASA rainfall. A real water balance also needs evapotranspiration, effective rainfall and soil storage. For the MVP, use categorical compatibility.

```
const waterDemandClass = (crop) => {
  const min = crop.water.fao_crop_water_need_min_mm;
  const max = crop.water.fao_crop_water_need_max_mm;

  if (min == null && max == null) return "UNKNOWN";
  if ((max ?? 0) >= 700) return "HIGH";
  if ((max ?? 0) >= 550) return "MEDIUM";
  return "LOW";
};

const evaluateWater = (crop, irrigationLevel) => {
  const demand = waterDemandClass(crop);

  if (demand === "UNKNOWN") {
    return { status: "UNKNOWN", score: null };
  }

  if (irrigationLevel === "high") {
    return { status: "PASS", score: 100 };
  }

  if (irrigationLevel === "medium") {
    return demand === "HIGH"
      ? { status: "CAUTION", score: 60 }
      : { status: "PASS", score: 85 };
  }

  if (demand === "HIGH") return { status: "POOR_MATCH", score: 25 };
  if (demand === "MEDIUM") return { status: "CAUTION", score: 55 };
  return { status: "PASS", score: 85 };
};
```

Important: The LOW/MEDIUM/HIGH cutoffs above are a coding framework, not agronomic truth. Your team should calibrate them with the exact crop rows and expert input before presenting them as validated

recommendations.

4.4 Rotation / Soil-Health Proxy

Use family diversity and legume status as transparent proxies. Do not claim that these simple rules calculate actual soil carbon or nitrogen balance.

```
const evaluateRotation = (currentCrop, candidateCrop) => {
  let score = 60;
  const reasons = [];

  const currentFamily = currentCrop.identity.family;
  const candidateFamily = candidateCrop.identity.family;

  if (currentFamily !== candidateFamily) {
    score += 20;
    reasons.push("Different crop family improves rotation diversity proxy");
  } else {
    score -= 20;
    reasons.push("Same crop family receives a diversity penalty");
  }

  if (candidateCrop.identity.is_legume_n_fixing === true) {
    score += 15;
    reasons.push("Legume receives biological-N-fixation proxy bonus");
  }

  return {
    score: Math.max(0, Math.min(100, score)),
    reasons
  };
};
```

5. Candidate Evaluation and Hard Filtering

Every crop should pass through the same candidate evaluator. Do not write separate ad-hoc code for wheat, mustard, lentil, etc. The dataset supplies the differences; the algorithm stays generic.

```
const evaluateCandidate = ({
  currentCrop,
  candidateCrop,
  plantingDate,
  turnaroundDays,
  seasonStartYear,
  soilPH,
  irrigationLevel,
  climateJson
}) => {
  const harvest = calculateHarvestRange(plantingDate, currentCrop);
  const fieldReady = calculateFieldReadyRange(harvest, turnaroundDays);
  const window = buildPlantingWindow(candidateCrop, seasonStartYear);
  const windowResult = evaluateWindow(fieldReady, window);

  const soil = evaluateSoilPH(soilPH, candidateCrop);
  const water = evaluateWater(candidateCrop, irrigationLevel);
  const rotation = evaluateRotation(currentCrop, candidateCrop);

  // Use the candidate's plausible planting date for climate context.
  const climateStart = windowResult.possiblePlantingDate;
  const climateEnd = addDays(climateStart, 30); // prototype early-establishment window
  const daily = getDailyRange(climateJson, climateStart, climateEnd);
  const temperature = evaluateTemperature(daily, candidateCrop);

  const hardRejected =
    windowResult.status === "OUTSIDE_SUPPORTED_WINDOW" ||
    soil.status === "OUTSIDE_TOLERANCE";

  return {
    cropId: candidateCrop.identity.crop_id,
    hardRejected,
    harvest,
    fieldReady,
    window,
    windowResult,
    soil,
    water,
    rotation,
    temperature
  };
};
```

Prototype design choice: The code above inspects the first 30 days after possible planting for early climate compatibility. You can later make this stage-specific using the crop calendar. Do not pretend a 30-day check is a full-season crop model.

5.1 Hard Constraints Must Come Before Ranking

- Planting window outside all documented support -> remove from normal recommendation list.
- Soil pH outside documented tolerance -> remove or require expert override.
- Missing data -> do not reject automatically; mark UNKNOWN and lower confidence.
- Temperature excursions -> warning/strong penalty unless you have validated failure rules.

- Water mismatch -> usually a penalty/caution rather than a hard reject in the MVP.

6. Ranking Feasible Alternatives

Once hard constraints have removed infeasible crops, convert the remaining factors into 0-100 project scores and combine them using the farmer's priorities.

Component	Example score rule	Purpose
Timeline	SAFE=100, COMPRESSED=60	Reward crops that still fit comfortably
Soil	Optimal=100, tolerated=60	Use known field pH when available
Water	Pass=85-100, caution=55-60, poor=25	Represent irrigation compatibility
Rotation	Family diversity + legume proxies	Represent soil/rotation quality
Climate risk	Penalize many outside-tolerance days / extreme historical context	Represent environmental risk
Income/yield	Use current price/cost if available; otherwise omit or label yield potential	Farmer economics

```
const normalizeWeights = (priorities) => {
  const total = Object.values(priorities).reduce((a, b) => a + b, 0);
  const out = {};
  for (const [k, v] of Object.entries(priorities)) out[k] = v / total;
  return out;
};

const timelineScore = (status) => {
  if (status === "SAFE") return 100;
  if (status === "COMPRESSED") return 60;
  return 0;
};

const climateRiskScore = (temperature) => {
  if (temperature.outsideToleranceDays === 0) return 90;
  if (temperature.outsideToleranceDays <= 2) return 65;
  return 35;
};

const scoreCandidate = (evaluation, priorities) => {
  const w = normalizeWeights(priorities);

  const scores = {
    timeline: timelineScore(evaluation.windowResult.status),
    soilHealth: evaluation.rotation.score,
    waterSaving: evaluation.water.score ?? 50,
    risk: climateRiskScore(evaluation.temperature),
    income: 50 // replace only when dynamic price/cost information exists
  };

  const final =
      scores.soilHealth * (w.soilHealth ?? 0)
    + scores.waterSaving * (w.waterSaving ?? 0)
    + scores.risk * (w.risk ?? 0)
    + scores.income * (w.income ?? 0);

  return { ...evaluation, componentScores: scores, finalScore: Math.round(final * 10) / 10 };
};
```

You can keep timeline as a hard feasibility gate and exclude it from the weighted score. That is often cleaner: first determine whether the crop still fits, then let farmer preferences rank only the feasible choices.

7. The Main decideRotation() Function

```
const decideRotation = ({ input, cropsById, climateJson }) => {
  validateInput(input, cropsById);

  const currentCrop = cropsById[input.currentCrop];
  const proposed = cropsById[input.proposedNextCrop];

  const common = {
    currentCrop,
    plantingDate: input.plantingDate,
    turnaroundDays: input.turnaroundDays,
    seasonStartYear: input.seasonStartYear,
    soilPH: input.soilPH,
    irrigationLevel: input.irrigation,
    climateJson
  };

  const originalPlan = evaluateCandidate({
    ...common,
    candidateCrop: proposed
  });

  const evaluations = Object.values(cropsById)
    .filter(c => c.identity.crop_id !== currentCrop.identity.crop_id)
    .map(candidateCrop =>
      evaluateCandidate({ ...common, candidateCrop })
    );

  const feasible = evaluations.filter(x => !x.hardRejected);

  const ranked = feasible
    .map(x => scoreCandidate(x, input.priorities))
    .sort((a, b) => b.finalScore - a.finalScore);

  return {
    originalPlan,
    alternatives: ranked,
    recommended: ranked[0] ?? null
  };
};
```

7.1 Recommended Input Object

```
const input = {
  currentCrop: "aman_br11",
  plantingDate: "2023-07-15",
  proposedNextCrop: "wheat_bari_gom33",
  seasonStartYear: 2023,
  turnaroundDays: 3,
  soilPH: 6.5,
  irrigation: "medium",
  priorities: {
    soilHealth: 5,
    waterSaving: 4,
    risk: 5,
    income: 2
  }
};
```

8. Worked Example: Aman BR11 -> Wheat BARI Gom-33

Item	Value from scenario / project dataset
Current crop	T. Aman rice - BR11
Transplanting date	2023-07-15
Reference duration	145 days
Estimated harvest	about 2023-12-07
Prototype turnaround input	3 days
Estimated field ready	about 2023-12-10
Next crop	Wheat - BARI Gom-33
Preferred sowing window	11-15 to 11-30
Supported late end	12-20

Window Guard reasoning:

Field ready: about Dec 10

↓

Preferred wheat window ended Nov 30

↓

Supported late window remains until Dec 20

↓

Result: COMPRESSED

The climate layer can then say that a chosen historical transition window was unusually wet/hot relative to the historical baseline, while the crop-tolerance layer separately checks whether actual temperatures remain inside wheat tolerance. The UI should explain both facts instead of merging them into one vague risk label.

```
{
  "originalPlan": {
    "rotation": "Aman BR11 -> Wheat BARI Gom-33",
    "status": "COMPRESSED",
    "estimatedHarvest": "2023-12-07",
    "fieldReady": "2023-12-10",
    "reason": "Field becomes ready after the preferred wheat window but before the documented supported-late end."
  }
}
```

8.1 Rotation GPS Example

If wheat is compressed, evaluate every other crop against the same field-ready date. A candidate is not an alternative merely because it has a high score; it must first survive the hard filters.

Candidate	Window status	Soil/water checks	Action
Wheat	COMPRESSED	Pass / caution	Keep
Mustard	Depends on current supported-late window	Evaluate	Keep only if feasible

Candidate	Window status	Soil/water checks	Action
Lentil	May already be outside optimum/support	Evaluate	Remove if window fails
Maize	May be outside Rabi window	Evaluate	Remove if window fails
Potato	Likely outside optimum if field releases in Dec	Evaluate	Remove if window fails

9. Recommended Output JSON

```
{
  "original_plan": {
    "crop": "wheat_bari_gom33",
    "window_status": "COMPRESSED",
    "harvest_range": {
      "earliest": "2023-12-07",
      "latest": "2023-12-07"
    },
    "field_ready_range": {
      "earliest": "2023-12-10",
      "latest": "2023-12-10"
    },
    "preferred_window": {
      "start": "2023-11-15",
      "end": "2023-11-30"
    },
    "supported_late_end": "2023-12-20"
  },
  "checks": {
    "soil": "OPTIMAL",
    "water": "PASS",
    "temperature": "PASS_WITH_CONTEXT",
    "rotation": "SAME_FAMILY_PENALTY"
  },
  "confidence": "HIGH",
  "alternatives": [
    {
      "crop_id": "...",
      "status": "SAFE",
      "final_score": 82.4,
      "reasons": ["...", "..."]
    }
  ]
}
```

10. Node / Express Integration

Your Node backend should load the two project-ready JSON files once when the server starts: the NASA climate JSON and the crop JSON. The crop JSON is already keyed by crop_id, so no CSV parser and no Python are required at runtime.

```
import fs from "fs";

const climateJson = JSON.parse(
  fs.readFileSync(
    "./data/rajshahi_nasa_project_ready.json",
    "utf8"
  )
);

const cropJson = JSON.parse(
  fs.readFileSync(
    "./data/fosol_cycle_rajshahi_crop_dataset.json",
    "utf8"
  )
);

// Already keyed by crop_id.
const cropsById = cropJson.crops;

app.post("/api/rotation/decide", (req, res) => {
  try {
    const result = decideRotation({
      input: req.body,
      cropsById,
      climateJson
    });

    res.json(result);
  } catch (error) {
    res.status(400).json({ error: error.message });
  }
});
```

10.1 React Request Example

```
const response = await fetch("/api/rotation/decide", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(input)
});

const decision = await response.json();

// Example UI fields:
console.log(decision.originalPlan.windowResult.status);
console.log(decision.alternatives);
console.log(decision.recommended);
```

11. Edge Cases Your Code Must Handle

Case	Correct behavior
Planting window crosses New Year	Attach end month/day to the next calendar year.

Case	Correct behavior
Field ready before preferred start	SAFE; actual planting date becomes preferred start.
No supported_late_end in dataset	Do not invent one. Use OUTSIDE/UNKNOWN based on confidence and expert rules.
Missing soil pH	Return soil check UNKNOWN; do not assume a value.
Missing water requirement	Water score UNKNOWN/neutral; lower confidence.
2026 climate record incomplete	Do not use incomplete periods as full historical baselines.
NASA date does not exist in JSON	Return missing-data warning; do not substitute another date silently.
Crop duration min != max	Preserve harvest/field-ready range; do not collapse to average by default.
Same crop appears in multiple season profiles	Use a specific crop_id/profile, not crop common name alone.
No feasible alternative	Return no supported option + reasons; never force a recommendation.

12. Confidence and Explanation Layer

Every recommendation should include reasons and caveats. This is important because several project rules have different evidence strength.

```
const buildExplanation = (evaluation, crop) => {
  const reasons = [];
  const cautions = [];

  reasons.push(`Window status: ${evaluation.windowResult.status}`);

  if (evaluation.soil.status === "OPTIMAL") {
    reasons.push("Field pH is inside the crop's optimal range");
  }

  if (evaluation.rotation.reasons) {
    reasons.push(...evaluation.rotation.reasons);
  }

  if (crop.planting_window.date_precision?.includes("APPROX")) {
    cautions.push("Planting dates are approximate encodings of week-level guidance");
  }

  if (crop.planting_window.hard_deadline_validated !== true) {
    cautions.push("Latest planting date is not validated as a biological hard deadline");
  }

  if (evaluation.water.status === "UNKNOWN") {
    cautions.push("Water compatibility could not be fully scored");
  }

  return {
    reasons,
    cautions,
    confidence: crop.decision_support.overall_data_confidence
  };
};
```

13. What NOT to Automate Yet

- Do not convert WET/HOT labels into exact harvest-delay days without local evidence.
- Do not treat NASA GWETTOP as a validated crop-specific field-readiness threshold.
- Do not calculate exact irrigation requirement from rainfall alone.
- Do not treat a preferred planting-window endpoint as a biological hard deadline unless validated.
- Do not claim that family diversity/legume bonuses calculate actual soil-carbon or nitrogen change.
- Do not use static reference yield as guaranteed farmer income.
- Do not allow an LLM to override hard crop constraints. AI may explain the result, not invent the agronomy.

14. Recommended Implementation Order

1. Load the crop JSON, assign cropJson.crops to cropsById, and verify all 8 profiles are available.
2. Load the NASA project-ready JSON.
3. Implement date helpers and cross-year planting-window logic.
4. Implement calculateHarvestRange() and calculateFieldReadyRange().
5. Implement evaluateWindow() and test SAFE / COMPRESSED / OUTSIDE cases.
6. Implement soil pH and basic water compatibility checks.
7. Implement climate daily-range extraction and temperature tolerance counts.
8. Implement evaluateRotation() family/legume proxy.
9. Implement evaluateCandidate() and unit-test each crop.
10. Implement hard filtering, scoring and farmer-priority ranking.
11. Add explanations/confidence to every output.
12. Expose one POST /api/rotation/decide endpoint and connect React.
13. Only after the core works, add more complex historical scenarios or expert-validated turnaround rules.

15. Minimal Test Cases

Test	Expected
Field ready inside wheat preferred window	SAFE
Field ready after Nov 30 but before Dec 20 for wheat profile	COMPRESSED
Field ready after supported late end	OUTSIDE_SUPPORTED_WINDOW
Field ready before crop preferred start	SAFE; possible planting date = preferred start
Soil pH outside tolerance	Hard reject / expert override needed
Missing soil pH	UNKNOWN, not reject
No feasible candidate after filters	Return empty alternatives with reasons
Farmer changes priority weights	Ranking may change; hard constraints must not

16. The Mental Model to Remember

Crop JSON: What does the crop require?
↓
NASA JSON: What environmental conditions occurred?
↓
Farmer input: What is the actual field plan?
↓
Window Guard: Does the next crop still fit in time?
↓
Hard constraints: Is it still physically/agronomically plausible?

↓
Rotation GPS: What other crops remain feasible?
↓
Farmer priorities: Which feasible option best matches the farmer?
↓
Explain the result with evidence, confidence and caveats

↓

NASA JSON: What environmental conditions occurred?

↓

Farmer input: What is the actual field plan?

↓

Window Guard: Does the next crop still fit in time?

↓

Hard constraints: Is it still physically/agronomically plausible?

↓

Rotation GPS: What other crops remain feasible?

↓

Farmer priorities: Which feasible option best matches the farmer?

↓

Explain the result with evidence, confidence and caveats

One-sentence algorithm: Constraints first, ranking second: determine whether a rotation still fits; if not, filter the remaining crops by evidence-backed constraints and then rank only the feasible options according to farmer priorities.

This guide is aligned to the current Fosol-Cycle MVP datasets: a project-ready NASA climate JSON plus a project-ready crop JSON generated from the human-maintained crop CSV. It intentionally marks unresolved agronomic rules instead of inventing precision. Update the CSV from validated research, regenerate the JSON, and keep the decision code generic.

17. Crop Data Build Pipeline: CSV -> Runtime JSON

Keep fosol_cycle_rajshahi_crop_dataset.csv as the research/editing master. Before the application runs, convert it once into fosol_cycle_rajshahi_crop_dataset.json. The decision engine then reads only JSON at runtime. This avoids CSV parsing in Node, preserves booleans and numbers as real JSON types, and groups related crop rules into clear nested sections.

Agriculture / research team edits the crop CSV
↓
Google Colab conversion script validates and converts values
↓
fosol_cycle_rajshahi_crop_dataset.json
↓
Node / Decision Engine loads cropJson.crops + NASA climate JSON

17.1 Recommended Runtime JSON Shape

```
{
  "metadata": { "crop_count": 8, "location_focus": "Rajshahi pilot, Bangladesh" },
  "crops": {
    "wheat_bari_gom33": {
      "identity": { "crop_id": "wheat_bari_gom33", "family": "Poaceae" },
      "planting_window": {
        "preferred_start_mmdd": "11-15",
        "preferred_end_mmdd": "11-30",
        "supported_late_end_mmdd": "12-20",
        "crosses_year": false
      },
      "duration": {
        "variety": { "min_days": 108, "max_days": 112 }
      },
      "environment": {
        "temperature": { "tolerance_min_c": 5, "tolerance_max_c": 27 },
        "soil": { "ph_optimal_min": 6, "ph_optimal_max": 7 }
      },
      "water": { "fao_crop_water_need_min_mm": 450, "fao_crop_water_need_max_mm": 650 },
      "decision_support": { "overall_data_confidence": "VERY_HIGH" }
    }
  }
}
```

17.2 Runtime Loading

```
const climateJson = JSON.parse(
  fs.readFileSync("./data/rajshahi_nasa_project_ready.json", "utf8")
);

const cropJson = JSON.parse(
  fs.readFileSync("./data/fosol_cycle_rajshahi_crop_dataset.json", "utf8")
);

const cropsById = cropJson.crops;
```

17.3 Update Rule

When the agriculture team corrects a planting window, variety duration, source, or other rule, edit the CSV first and regenerate the JSON. Do not hand-edit both copies independently. Treat CSV as the research source-of-truth and JSON as the generated runtime artifact.

Revision note: v2 switches the runtime crop data source from CSV parsing to a typed, nested crop JSON while keeping the CSV as the editable research master.